



Création et utilisation de la base de données

Pierre-Antoine Morin



Laplace Immo

Contexte du projet

- **Stratégie**
Se démarquer de la concurrence
en exploitant le pouvoir des données
- **Objectif**
Mieux prévoir le prix de vente
des biens immobiliers
- **Proof Of Concept**
1^{er} semestre 2020



Stratégie de sauvegarde et conformité RGPD

5 grands principes



Mise en œuvre

- Documentation : dictionnaire des données, schéma relationnel normalisé
- Données publiques, anonymisées et filtrées : conservation des informations géographiques et immobilières uniquement
- Base de données hébergée sur un serveur interne à l'entreprise, accès restreint aux utilisateurs agréés

Données initiales

- 3 fichiers CSV, données publiques
- Demandes de valeurs foncières du 1^{er} semestre de 2020
 - Source : DGFIP
- Données des communes avec population
 - Source : INSEE
- Référentiel géographique français
 - Source : data.gouv

Dictionnaire des données

REGION							
Colonne	Type	Taille	NOT NULL	CHECK	PRIMARY KEY	FOREIGN KEY	Description
reg_code	VARCHAR	2	X		X		Code région (2 chiffres)
reg_nom	VARCHAR	50	X				Nom de la région

DEPARTEMENT							
Colonne	Type	Taille	NOT NULL	CHECK	PRIMARY KEY	FOREIGN KEY	Description
dep_code	VARCHAR	3	X		X		Code département (2 ou 3 chiffres sauf Corse 2A/2B)
dep_nom	VARCHAR	50	X				Nom du département
reg_code	VARCHAR	2	X			Région	Code région (2 chiffres)

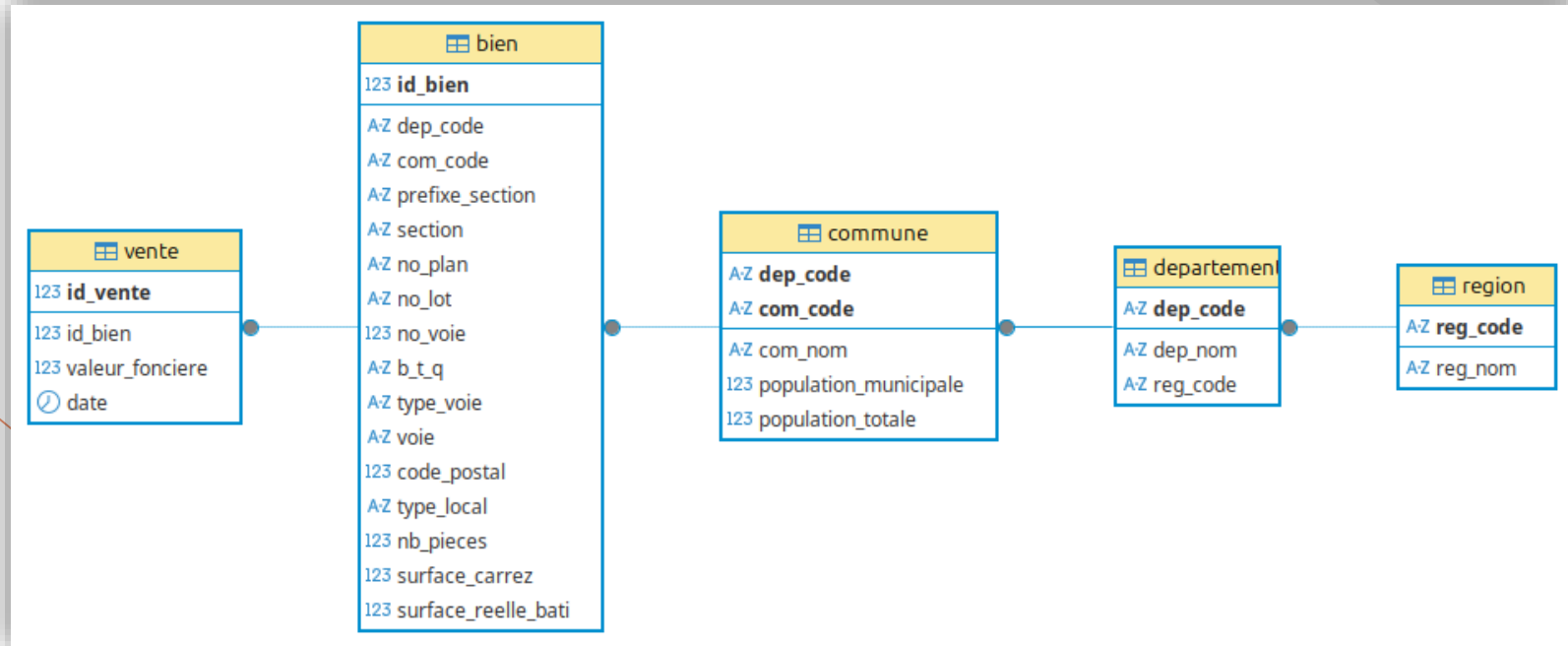
COMMUNE							
Colonne	Type	Taille	NOT NULL	CHECK	PRIMARY KEY	FOREIGN KEY	Description
dep_code	VARCHAR	3	X		X	Département	Code département (2 ou 3 chiffres sauf Corse 2A/2B)
com_code	VARCHAR	3	X				Code commune (3 chiffres)
com_nom	VARCHAR	50	X				Nom de la commune
population_municipale	INT		X	>= 0			Population municipale (source INSEE)
population_totale	INT		X	>= population_municipale			Population totale (source INSEE)

Dictionnaire des données

BIEN							
Colonne	Type	Taille	NOT NULL	CHECK	PRIMARY KEY	FOREIGN KEY	Description
id_bien	SERIAL		X		X		Identifiant du bien (clé artificielle)
dep_code	VARCHAR	3	X			Commune	Code département (2 ou 3 chiffres sauf Corse 2A/2B)
com_code	VARCHAR	3	X				Code commune (3 chiffres)
prefixe_section	VARCHAR	3	X				Élément de référence cadastrale
section	VARCHAR	2	X				Élément de référence cadastrale
no_plan	VARCHAR	4	X				Élément de référence cadastrale
no_lot	VARCHAR	7	X				Numéro de lot de copropriété (entier, parfois suffixe A/B/N)
no_voie	INT			> 0			Numéro dans la voie
b_t_q	VARCHAR	1					Indice de répétition
type_voie	VARCHAR	4					Type de voie (rue, avenue, ...)
voie	VARCHAR	50					Libellé de la voie
code_postal	INT			(>= 1000) AND (< 100000)			Code postal
type_local	TYPE_LOCAL		X				ENUM('Maison', 'Appartement')
nb_pieces	INT		X	>= 0			Nombre de pièces
surface_carrez	FLOAT		X	> 0			Surface carrez (m²)
surface_reelle_bati	INT		X	> 0			Surface réelle arrondie au m² inférieur

VENTE							
Colonne	Type	Taille	NOT NULL	CHECK	PRIMARY KEY	FOREIGN KEY	Description
id_vente	SERIAL		X		X		Identifiant de la vente (clé artificielle)
id_bien	SERIAL		X			Bien	Identifiant du bien (clé artificielle)
valeur_fonciere	FLOAT			> 0			Montant de la transaction (€)
date	DATE		X				Date de la transaction

Schéma relationnel normalisé

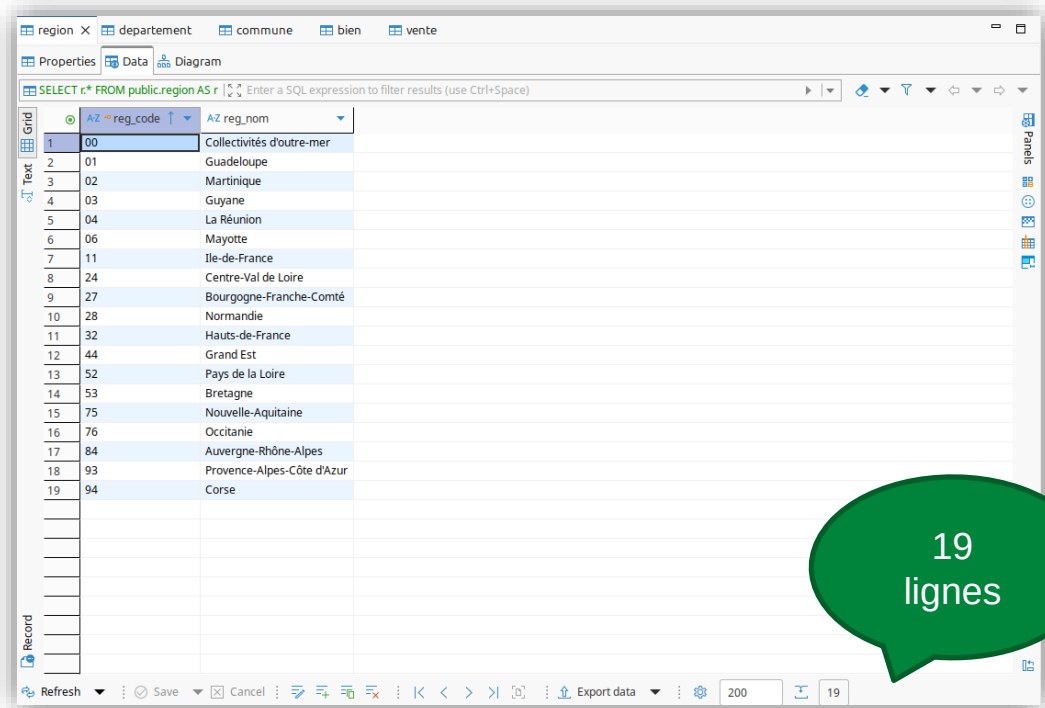


Chargement de la base de données

- Préparation des fichiers CSV
 - Suppression/renommage de colonnes
- Création de la base de données (PostgreSQL)
- Assistant Import CSV (DBEaver)
 - Tous les champs au format texte
- Création du schéma relationnel (CREATE TABLE)
- Remplissage des tables (INSERT INTO)
 - 1. Région CSV référentiel géographique
 - 2. Département CSV référentiel géographique
 - 3. Commune CSV données communes
 - 4. Bien CSV demande de valeurs foncières
 - 5. Vente CSV demande de valeurs foncières

Base de données chargée

Table Région



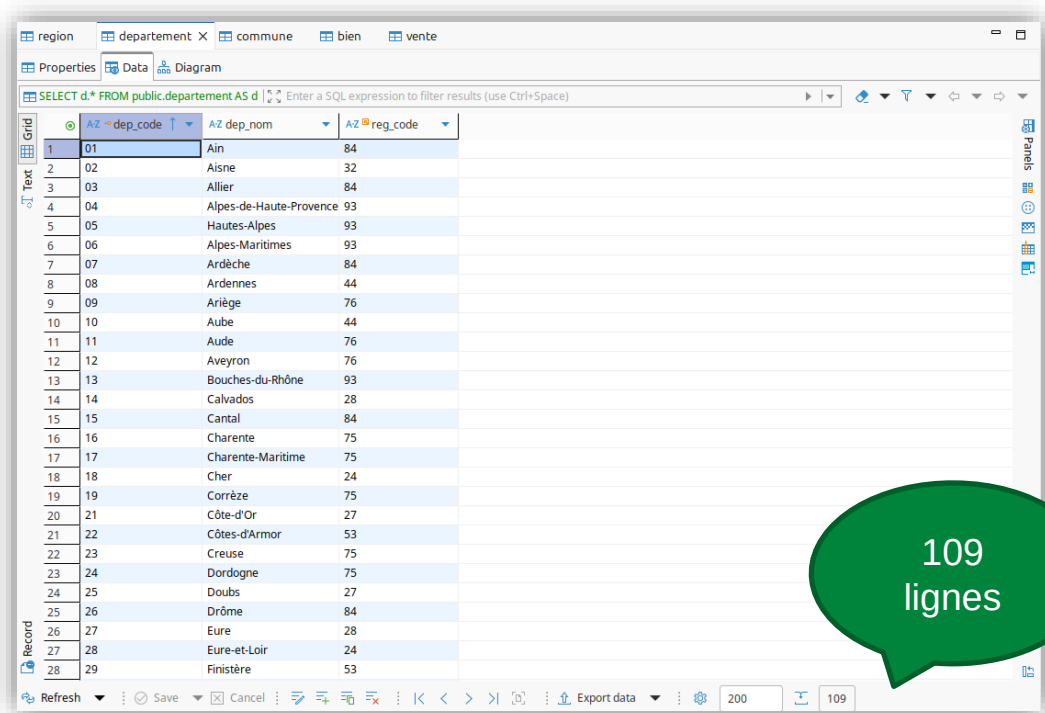
The screenshot shows a database application window with the 'region' table selected. The table has two columns: 'reg_code' and 'reg_nom'. The data is displayed in a grid view with 19 rows. A green speech bubble with the text '19 lignes' points to the bottom of the table.

	reg_code	reg_nom
1	00	Collectivités d'outre-mer
2	01	Guadeloupe
3	02	Martinique
4	03	Guyane
5	04	La Réunion
6	06	Mayotte
7	11	Ile-de-France
8	24	Centre-Val de Loire
9	27	Bourgogne-Franche-Comté
10	28	Normandie
11	32	Hauts-de-France
12	44	Grand Est
13	52	Pays de la Loire
14	53	Bretagne
15	75	Nouvelle-Aquitaine
16	76	Occitanie
17	84	Auvergne-Rhône-Alpes
18	93	Provence-Alpes-Côte d'Azur
19	94	Corse

19
lignes

Base de données chargée

Table Département



region | departement X | commune | bien | vente

Properties | Data | Diagram

SELECT d* FROM public.departement AS d | Enter a SQL expression to filter results (use Ctrl+Space)

	dep_code	dep_nom	reg_code
1	01	Ain	84
2	02	Aisne	32
3	03	Allier	84
4	04	Alpes-de-Haute-Provence	93
5	05	Hautes-Alpes	93
6	06	Alpes-Maritimes	93
7	07	Ardèche	84
8	08	Ardennes	44
9	09	Ariège	76
10	10	Aube	44
11	11	Aude	76
12	12	Aveyron	76
13	13	Bouches-du-Rhône	93
14	14	Calvados	28
15	15	Cantal	84
16	16	Charente	75
17	17	Charente-Maritime	75
18	18	Cher	24
19	19	Corrèze	75
20	21	Côte-d'Or	27
21	22	Côtes-d'Armor	53
22	23	Creuse	75
23	24	Dordogne	75
24	25	Doubs	27
25	26	Drôme	84
26	27	Eure	28
27	28	Eure-et-Loir	24
28	29	Finistère	53

Refresh | Save | Cancel | Export data | 200 | 109

109 lignes

Base de données chargée

Table Commune

region departement commune X bien vente

Properties Data Diagram

SELECT c.* FROM public.commune AS c ORDER BY ... Enter a SQL expression to filter results (use Ctrl+Space)

	AZ dep_code	AZ com_code	AZ com_nom	123 population_municipale	123 population_totale
1	01	001	L'Abergement-Clémenciat	779	798
2	01	002	L'Abergement-de-Varey	256	257
3	01	004	Ambérieu-en-Bugey	14,134	14,514
4	01	005	Ambérieux-en-Dombes	1,751	1,776
5	01	006	Ambléon	112	118
6	01	007	Ambronay	2,800	2,915
7	01	008	Ambutrix	762	777
8	01	009	Anderet-et-Condou	326	335
9	01	010	Angletfort	1,105	1,122
10	01	011	Apremont	368	379
11	01	012	Aranc	329	333
12	01	013	Arandas	143	147
13	01	014	Arbent	3,349	3,442
14	01	015	Arbois en Bugey	650	667
15	01	016	Arbigny	460	469
16	01	017	Argis	457	473
17	01	019	Armix	27	31
18	01	021	Ars-sur-Formans	1,457	1,485
19	01	022	Artemare	1,249	1,267
20	01	023	Asnières-sur-Saône	76	76
21	01	024	Attignat	3,251	3,316
22	01	025	Bâgé-Dommartin	4,065	4,171
23	01	026	Bâgé-le-Châtel	973	1,041
24	01	027	Balan	2,563	2,615
25	01	028	Baneins	617	634
26	01	029	Beaupont	685	693
27	01	030	Beauregard	829	844
28	01	031	Bellignat	3,652	3,886

Refresh Save Cancel Export data 200 34,991

34 991 lignes

Base de données chargée

Table Bien

```
SELECT COUNT(DISTINCT (dep_code, com_code))  
FROM bien
```

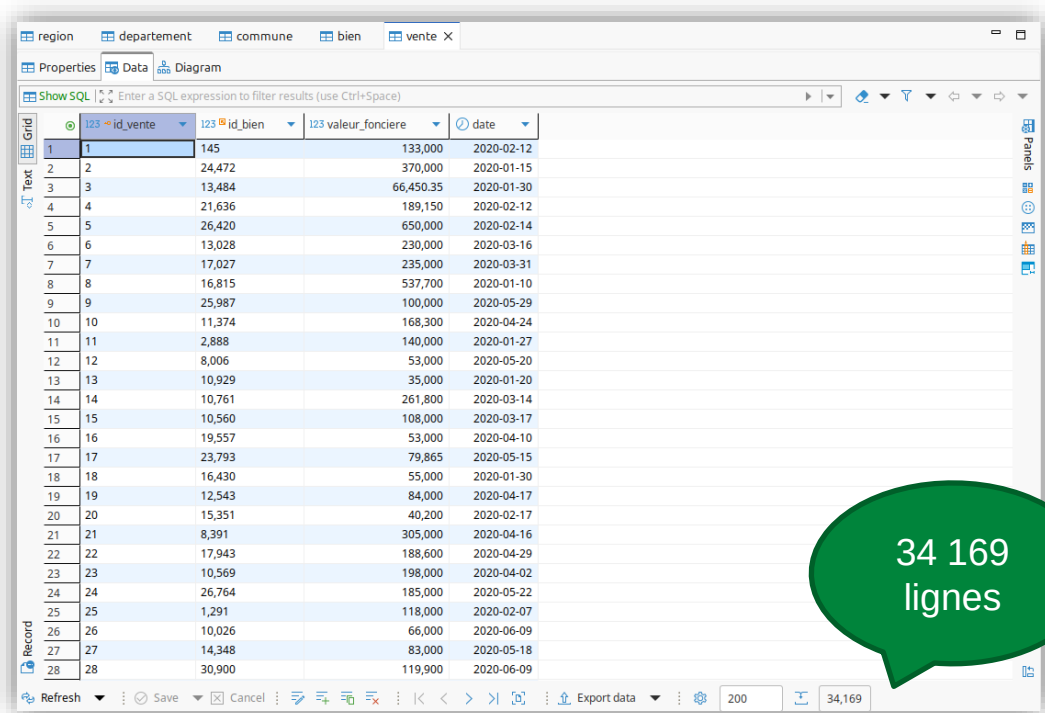
14	14	13	202	808	D	103	408
15	15	91	174		AH	57	13
16	16	91	275		AB	370	9
17	17	35	288		AB	1	10
18	18	69	384		AL	10	49
19	19	94	077		AP	210	21
20	20	78	498		AT	23	2
21	21	75	118		AI	18	412
22	22	75	112		BR	39	23
23	23	75	116		DJ	18	29
24	24	78	383		AC	86	21
25	25	31	555	845	AT	160	56
26	26	42	187		AX	604	27
27	27	44	109		HW	72	15
28	28	93	001		BH	83	255

dans
3 125
communes

34 169
lignes

Base de données chargée

Table Vente



The screenshot shows a database management interface with the 'vente' table selected. The table has four columns: 'id_vente', 'id_bien', 'valeur_fonciere', and 'date'. The data is displayed in a grid view, showing records 1 through 28. A green speech bubble in the bottom right corner of the grid area indicates the total number of records: 34 169 lignes.

	id_vente	id_bien	valeur_fonciere	date
1	1	145	133,000	2020-02-12
2	2	24,472	370,000	2020-01-15
3	3	13,484	66,450.35	2020-01-30
4	4	21,636	189,150	2020-02-12
5	5	26,420	650,000	2020-02-14
6	6	13,028	230,000	2020-03-16
7	7	17,027	235,000	2020-03-31
8	8	16,815	537,700	2020-01-10
9	9	25,987	100,000	2020-05-29
10	10	11,374	168,300	2020-04-24
11	11	2,888	140,000	2020-01-27
12	12	8,006	53,000	2020-05-20
13	13	10,929	35,000	2020-01-20
14	14	10,761	261,800	2020-03-14
15	15	10,560	108,000	2020-03-17
16	16	19,557	53,000	2020-04-10
17	17	23,793	79,865	2020-05-15
18	18	16,430	55,000	2020-01-30
19	19	12,543	84,000	2020-04-17
20	20	15,351	40,200	2020-02-17
21	21	8,391	305,000	2020-04-16
22	22	17,943	188,600	2020-04-29
23	23	10,569	198,000	2020-04-02
24	24	26,764	185,000	2020-05-22
25	25	1,291	118,000	2020-02-07
26	26	10,026	66,000	2020-06-09
27	27	14,348	83,000	2020-05-18
28	28	30,900	119,900	2020-06-09

34 169
lignes



Requêtes SQL et résultats

Nombre total d'appartements vendus (2020, semestre 1)

Requête 1

```
SELECT COUNT(*) AS "nb ventes (S1 2020, appartements)"  
FROM bien NATURAL JOIN vente  
WHERE type_local = 'Appartement'  
      AND "date" >= make_date(2020, 1, 1)  
      AND "date" <  make_date(2020, 7, 1)
```



31 378
ventes

Nombre de ventes d'appartements par région (2020, semestre 1)

Requête 2

```
SELECT
    reg_nom,
    COUNT(id_vente) AS "nb ventes (S1 2020, appartements)"
FROM region NATURAL LEFT JOIN departement
    NATURAL LEFT JOIN commune
    NATURAL LEFT JOIN bien
    NATURAL LEFT JOIN vente
WHERE id_vente IS NULL OR (
    type_local = 'Appartement'
    AND "date" >= make_date(2020, 1, 1)
    AND "date" < make_date(2020, 7, 1)
)
GROUP BY reg_code, reg_nom
ORDER BY COUNT(id_vente) DESC, reg_nom ASC
```


Nombre de ventes d'appartements par région (2020, semestre 1)

Requête 2

The screenshot shows a database query results window titled 'Résultats'. The query is `select reg_nom, count(id_vente) as 'nb ven'`. The results are displayed in a table with 19 rows, representing different French regions. The first row is 'Ile-de-France' with 13,995 sales. The last row is 'Mayotte' with 0 sales. The table is sorted by the number of sales in descending order.

	AZ reg_nom	123 nb ventes (S1 2020, appartements)
1	Ile-de-France	13,995
2	Provence-Alpes-Côte d'Azur	3,649
3	Auvergne-Rhône-Alpes	3,253
4	Nouvelle-Aquitaine	1,932
5	Occitanie	1,640
6	Pays de la Loire	1,357
7	Hauts-de-France	1,254
8	Grand Est	984
9	Bretagne	983
10	Normandie	862
11	Centre-Val de Loire	696
12	Bourgogne-Franche-Comté	376
13	Corse	223
14	Martinique	94
15	La Réunion	44
16	Guyane	34
17	Guadeloupe	2
18	Collectivités d'outre-mer	0
19	Mayotte	0

Proportion des ventes d'appartements par le nombre de pièces

Requête 3

```
SELECT
  nb_pieces,
  ROUND(100.0 * COUNT(id_vente) / (
    SELECT COUNT(id_vente)
    FROM bien NATURAL JOIN vente
    WHERE type_local = 'Appartement'
  ), 2) AS "pourcentage des ventes (appartements)"
FROM bien NATURAL LEFT JOIN vente
WHERE type_local = 'Appartement'
GROUP BY nb_pieces
ORDER BY nb_pieces ASC
```

Proportion des ventes d'appartements par le nombre de pièces

Requête 3

Résultats X

select nb_pieces, round(100.0 * count(id_v) Enter a SQL expression to filter results (use Ctrl+Space)

	123 nb_pieces	123 pourcentage des ventes (appartements)
1	0	0.1
2	1	21.48
3	2	31.18
4	3	28.57
5	4	14.21
6	5	3.55
7	6	0.65
8	7	0.17
9	8	0.05
10	9	0.03
11	10	0.01
12	11	0

Refresh Save Cancel Export data 200 12

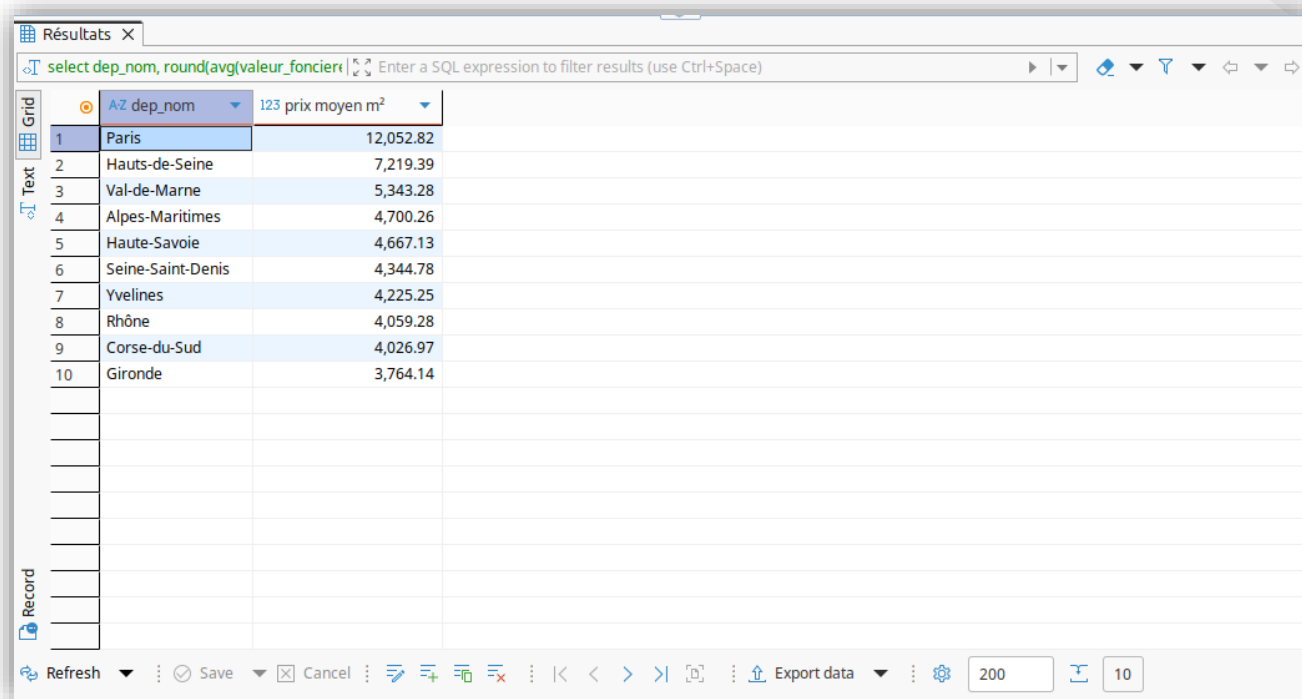
10 départements où le prix du mètre carré est le plus élevé

Requête 4

```
SELECT
    dep_nom,
    ROUND(AVG(valeur_fonciere / surface_carrez)::numeric, 2)
        AS "prix moyen m²"
FROM departement NATURAL JOIN commune
        NATURAL JOIN bien
        NATURAL JOIN vente
-- WHERE valeur_fonciere IS NOT NULL
GROUP BY dep_code, dep_nom
ORDER BY
    AVG(valeur_fonciere / surface_carrez) DESC,
    dep_nom ASC
LIMIT 10
```

10 départements où le prix du mètre carré est le plus élevé

Requête 4



The screenshot shows a database query results window titled "Résultats". The SQL query entered is `select dep_nom, round(avg(valeur_foncier)`. The results are displayed in a table with two columns: "dep_nom" and "prix moyen m²". The table lists the top 10 departments by average price per square meter, with Paris having the highest price at 12,052.82.

	AZ dep_nom	123 prix moyen m²
1	Paris	12,052.82
2	Hauts-de-Seine	7,219.39
3	Val-de-Marne	5,343.28
4	Alpes-Maritimes	4,700.26
5	Haute-Savoie	4,667.13
6	Seine-Saint-Denis	4,344.78
7	Yvelines	4,225.25
8	Rhône	4,059.28
9	Corse-du-Sud	4,026.97
10	Gironde	3,764.14

Prix moyen du mètre carré d'une maison en Île-de-France

Requête 5

```
SELECT
  ROUND(AVG(valeur_fonciere / surface_carrez)::numeric, 2)
  AS "prix moyen m² (maisons, Île-de-France)"
FROM region NATURAL JOIN departement
      NATURAL JOIN commune
      NATURAL JOIN bien
      NATURAL JOIN vente
WHERE reg_nom = 'Ile-de-France' AND type_local = 'Maison'
```



3 745,09
€/m²

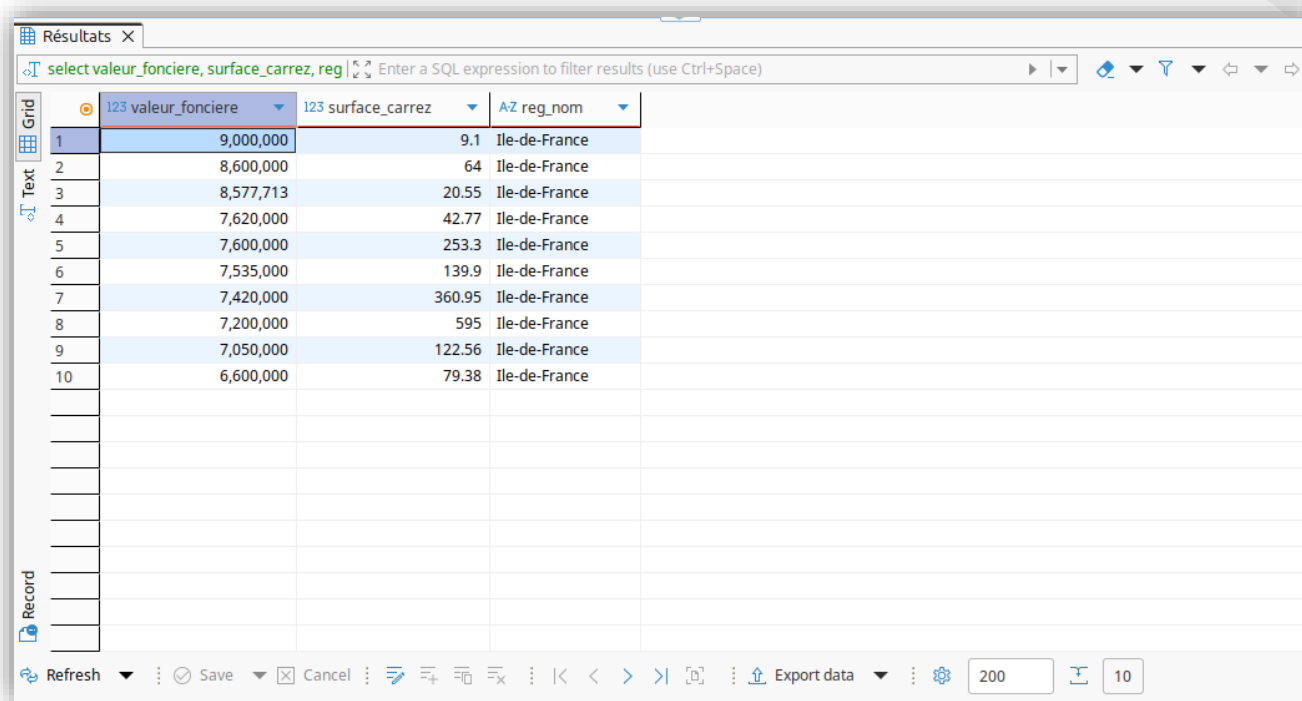
Liste de 10 appartements les plus chers avec région et superficie (m²)

Requête 6

```
SELECT
    valeur_fonciere,
    surface_carrez,
    reg_nom
FROM region NATURAL JOIN departement
        NATURAL JOIN commune
        NATURAL JOIN bien
        NATURAL JOIN vente
WHERE type_local = 'Appartement'
    AND valeur_fonciere IS NOT NULL
ORDER BY valeur_fonciere DESC
LIMIT 10
```

Liste de 10 appartements les plus chers avec région et superficie (m²)

Requête 6



The screenshot shows a database query results window titled "Résultats". The query bar contains the SQL expression: `select valeur_fonciere, surface_carrez, reg`. The results are displayed in a table with 4 columns: an index, `valeur_fonciere`, `surface_carrez`, and `reg_nom`. The table contains 10 rows of data, all from the "Ile-de-France" region. The interface includes a sidebar with "Grid" and "Text" views, and a bottom toolbar with options like "Refresh", "Save", "Cancel", and "Export data".

	123 valeur_fonciere	123 surface_carrez	AZ reg_nom
1	9,000,000	9.1	Ile-de-France
2	8,600,000	64	Ile-de-France
3	8,577,713	20.55	Ile-de-France
4	7,620,000	42.77	Ile-de-France
5	7,600,000	253.3	Ile-de-France
6	7,535,000	139.9	Ile-de-France
7	7,420,000	360.95	Ile-de-France
8	7,200,000	595	Ile-de-France
9	7,050,000	122.56	Ile-de-France
10	6,600,000	79.38	Ile-de-France

Taux d'évolution du nombre de ventes (2020, trimestres 1 et 2)

Requête 7

```
CREATE VIEW vente_t1_2020 AS
SELECT *
FROM vente
WHERE "date" >= make_date(2020, 1, 1)
      AND "date" < make_date(2020, 4, 1)
```

```
CREATE VIEW vente_t2_2020 AS
SELECT *
FROM vente
WHERE "date" >= make_date(2020, 4, 1)
      AND "date" < make_date(2020, 7, 1)
```

16 776
lignes

17 393
lignes

Taux d'évolution du nombre de ventes (2020, trimestres 1 et 2)

Requête 7

```
CREATE FUNCTION difference_pourcentage (x numeric, y numeric)
RETURNS numeric AS $$
    SELECT 100.0 * (y - x) / x
$$ LANGUAGE SQL
```

```
SELECT ROUND(difference_pourcentage(
    (SELECT COUNT(*) FROM vente_t1_2020),
    (SELECT COUNT(*) FROM vente_t2_2020)
), 2) AS "evolution ventes entre T1 et T2 2020 (%)"
```



+ 3,68 %

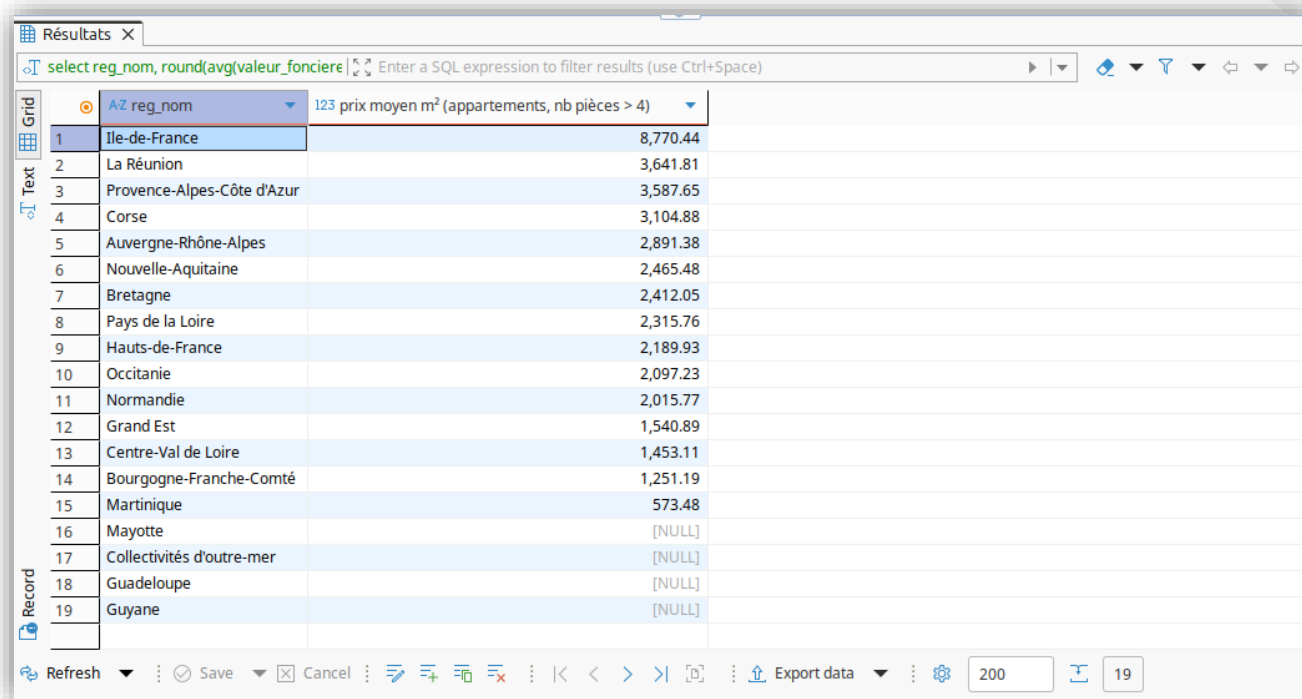
Classement des régions par prix au mètre carré (appartements > 4 pièces)

Requête 8

```
SELECT
  reg_nom,
  ROUND(AVG(valeur_fonciere / surface_carrez)::numeric, 2)
    AS "prix moyen m² (appartements, nb pièces > 4)"
FROM region NATURAL LEFT JOIN departement
      NATURAL LEFT JOIN commune
      NATURAL LEFT JOIN bien
      NATURAL LEFT JOIN vente
WHERE valeur_fonciere IS NULL
   OR (type_local = 'Appartement' AND nb_pieces > 4)
GROUP BY reg_code, reg_nom
ORDER BY
  (CASE WHEN COUNT(id_vente) = 0 THEN 1 ELSE 0 END) ASC,
  AVG(valeur_fonciere / surface_carrez) DESC
```

Classement des régions par prix au mètre carré (appartements > 4 pièces)

Requête 8



The screenshot shows a database query results window titled "Résultats". The query is: `select reg_nom, round(avg(valeur_fonciere`. The results are displayed in a table with 2 columns: "reg_nom" and "prix moyen m² (appartements, nb pièces > 4)". The table lists 19 regions, sorted by price in descending order. The first region is Ile-de-France with a price of 8,770.44. The last region is Guyane with a price of [NULL].

	reg_nom	123 prix moyen m² (appartements, nb pièces > 4)
1	Ile-de-France	8,770.44
2	La Réunion	3,641.81
3	Provence-Alpes-Côte d'Azur	3,587.65
4	Corse	3,104.88
5	Auvergne-Rhône-Alpes	2,891.38
6	Nouvelle-Aquitaine	2,465.48
7	Bretagne	2,412.05
8	Pays de la Loire	2,315.76
9	Hauts-de-France	2,189.93
10	Occitanie	2,097.23
11	Normandie	2,015.77
12	Grand Est	1,540.89
13	Centre-Val de Loire	1,453.11
14	Bourgogne-Franche-Comté	1,251.19
15	Martinique	573.48
16	Mayotte	[NULL]
17	Collectivités d'outre-mer	[NULL]
18	Guadeloupe	[NULL]
19	Guyane	[NULL]

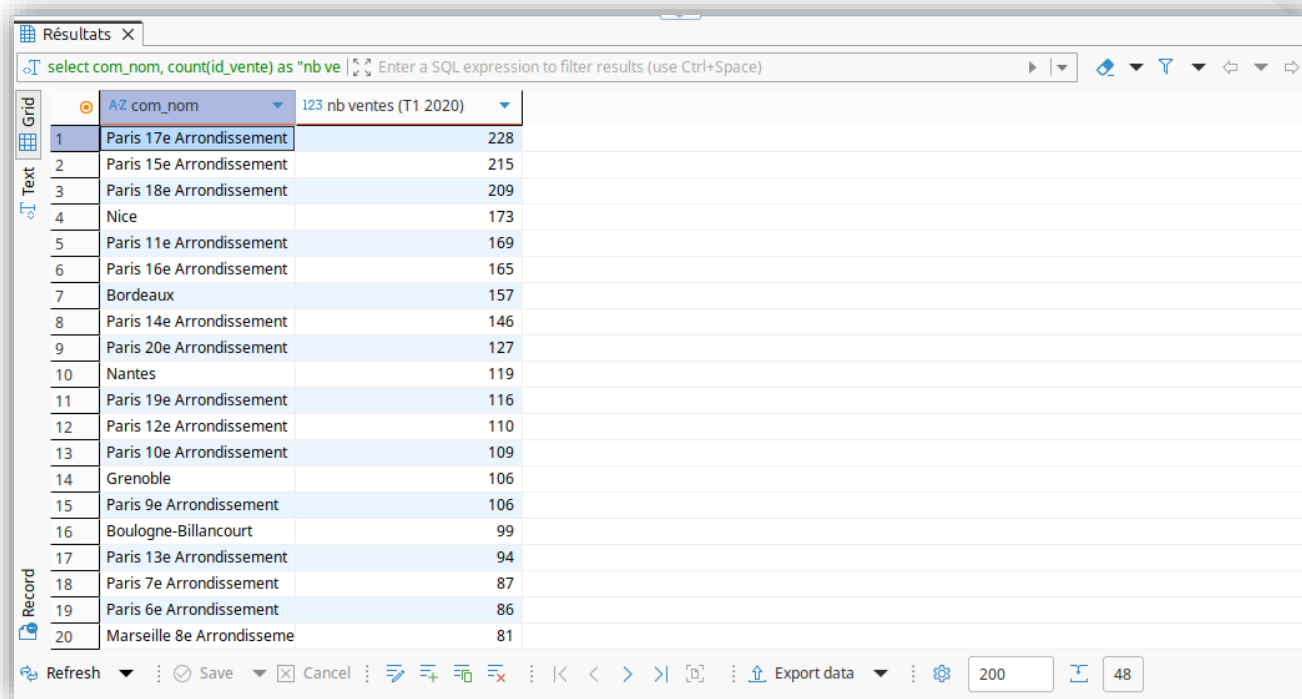
Communes avec au moins 50 ventes (2020, trimestre 1)

Requête 9

```
SELECT
  com_nom,
  COUNT(id_vente) AS "nb ventes (T1 2020)"
FROM commune NATURAL JOIN bien
      NATURAL JOIN vente_t1_2020
GROUP BY dep_code, com_code, com_nom
HAVING COUNT(id_vente) >= 50
ORDER BY COUNT(id_vente) DESC
```

Communes avec au moins 50 ventes (2020, trimestre 1)

Requête 9



The screenshot shows a software interface for viewing query results. At the top, a tab labeled 'Résultats' is active. Below it, a SQL query is entered: `select com_nom, count(id_vente) as "nb ve"`. The results are displayed in a table with two columns: 'com_nom' and 'nb ventes (T1 2020)'. The table lists 20 communes, sorted by sales count in descending order. The first commune, Paris 17e Arrondissement, has 228 sales. The last commune, Marseille 8e Arrondissement, has 81 sales. The interface includes a sidebar with 'Grid', 'Text', and 'Record' views, and a bottom toolbar with buttons for 'Refresh', 'Save', 'Cancel', and 'Export data'.

	AZ com_nom	123 nb ventes (T1 2020)
1	Paris 17e Arrondissement	228
2	Paris 15e Arrondissement	215
3	Paris 18e Arrondissement	209
4	Nice	173
5	Paris 11e Arrondissement	169
6	Paris 16e Arrondissement	165
7	Bordeaux	157
8	Paris 14e Arrondissement	146
9	Paris 20e Arrondissement	127
10	Nantes	119
11	Paris 19e Arrondissement	116
12	Paris 12e Arrondissement	110
13	Paris 10e Arrondissement	109
14	Grenoble	106
15	Paris 9e Arrondissement	106
16	Boulogne-Billancourt	99
17	Paris 13e Arrondissement	94
18	Paris 7e Arrondissement	87
19	Paris 6e Arrondissement	86
20	Marseille 8e Arrondissement	81

Différence (%) de prix au mètre carré entre appartements de 2 ou 3 pièces

Requête 10

```
SELECT ROUND(difference_pourcentage(  
  (  
    SELECT AVG(valeur_fonciere / surface_carrez)::numeric  
    FROM bien NATURAL JOIN vente  
    WHERE type_local = 'Appartement' AND nb_pieces = 2  
  ),  
  (  
    SELECT AVG(valeur_fonciere / surface_carrez)::numeric  
    FROM bien NATURAL JOIN vente  
    WHERE type_local = 'Appartement' AND nb_pieces = 3  
  )  
, 2) AS "différence (%) prix moyen m² (appt 2/3 pièces)"
```



- 12,40 %

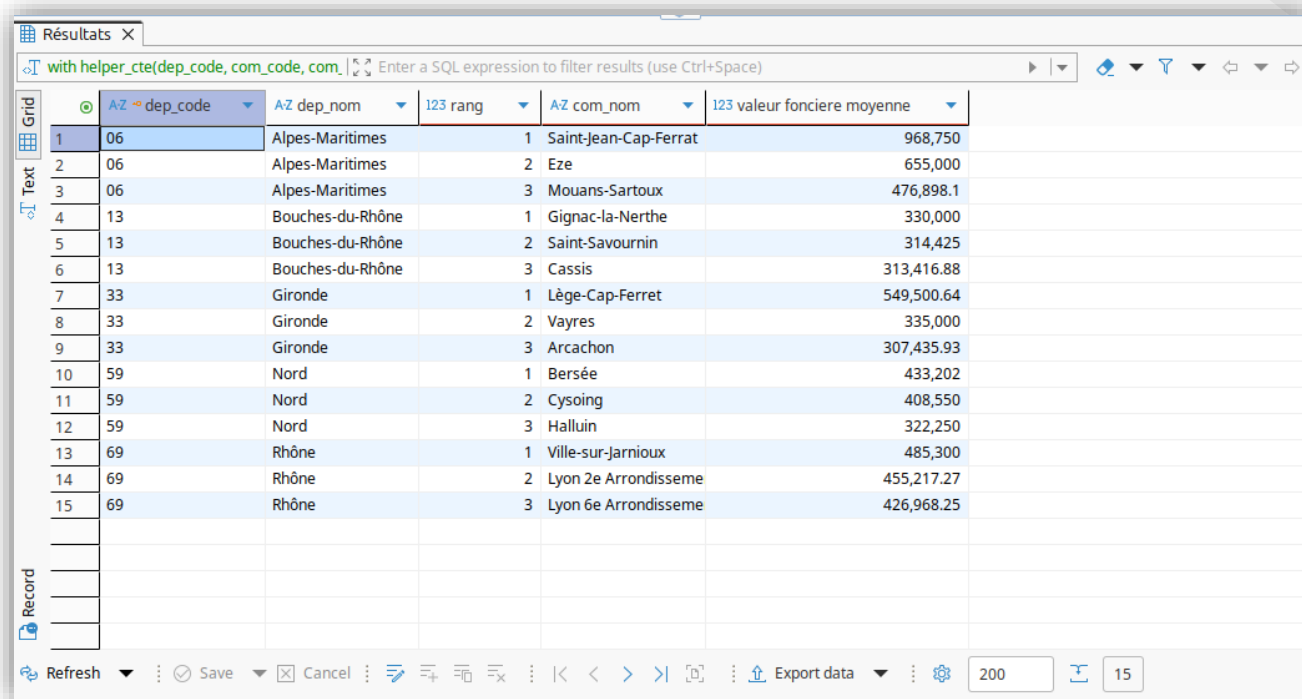
Top 3 des communes par valeurs foncières moyennes (départements 6, 13, 33, 59 et 69)

Requête 11

```
WITH helper_cte(dep_code, com_code, com_nom, valeur_fonciere_moyenne, rang) AS
(
    SELECT
        dep_code,
        com_code,
        com_nom,
        AVG(valeur_fonciere),
        RANK() OVER (PARTITION BY dep_code ORDER BY AVG(valeur_fonciere) DESC)
    FROM commune NATURAL JOIN bien NATURAL JOIN vente
    WHERE dep_code IN ('06', '13', '33', '59', '69')
    GROUP BY dep_code, com_code
)
SELECT dep_code, dep_nom, rang, com_nom,
       ROUND(valeur_fonciere_moyenne::numeric, 2) AS "valeur fonciere moyenne"
FROM departement NATURAL JOIN helper_cte
WHERE rang <= 3
ORDER BY dep_code ASC, rang ASC
```


Top 3 des communes par valeurs foncières moyennes (départements 6, 13, 33, 59 et 69)

Requête 11



The screenshot shows a software interface for viewing query results. At the top, a tab labeled 'Résultats' is active. Below it, a text input field contains the query: 'with helper_cte(dep_code, com_code, com_'. To the right of the input field are navigation icons. The main area displays a table with 6 columns: a row number, 'dep_code', 'dep_nom', 'rang', 'com_nom', and 'valeur fonciere moyenne'. The table lists the top 3 communes for each of five departments (6, 13, 33, 59, 69). On the left side of the table, there are vertical labels 'Grid', 'Text', and 'Record' with corresponding icons. At the bottom, there is a toolbar with buttons for 'Refresh', 'Save', 'Cancel', and 'Export data', along with pagination controls showing '200' and '15'.

	dep_code	dep_nom	123 rang	com_nom	123 valeur fonciere moyenne
1	06	Alpes-Maritimes	1	Saint-Jean-Cap-Ferrat	968,750
2	06	Alpes-Maritimes	2	Eze	655,000
3	06	Alpes-Maritimes	3	Mouans-Sartoux	476,898.1
4	13	Bouches-du-Rhône	1	Gignac-la-Nerthe	330,000
5	13	Bouches-du-Rhône	2	Saint-Savournin	314,425
6	13	Bouches-du-Rhône	3	Cassis	313,416.88
7	33	Gironde	1	Lège-Cap-Ferret	549,500.64
8	33	Gironde	2	Vayres	335,000
9	33	Gironde	3	Arcachon	307,435.93
10	59	Nord	1	Bersée	433,202
11	59	Nord	2	Cysoing	408,550
12	59	Nord	3	Halluin	322,250
13	69	Rhône	1	Ville-sur-Jarnioux	485,300
14	69	Rhône	2	Lyon 2e Arrondissement	455,217.27
15	69	Rhône	3	Lyon 6e Arrondissement	426,968.25

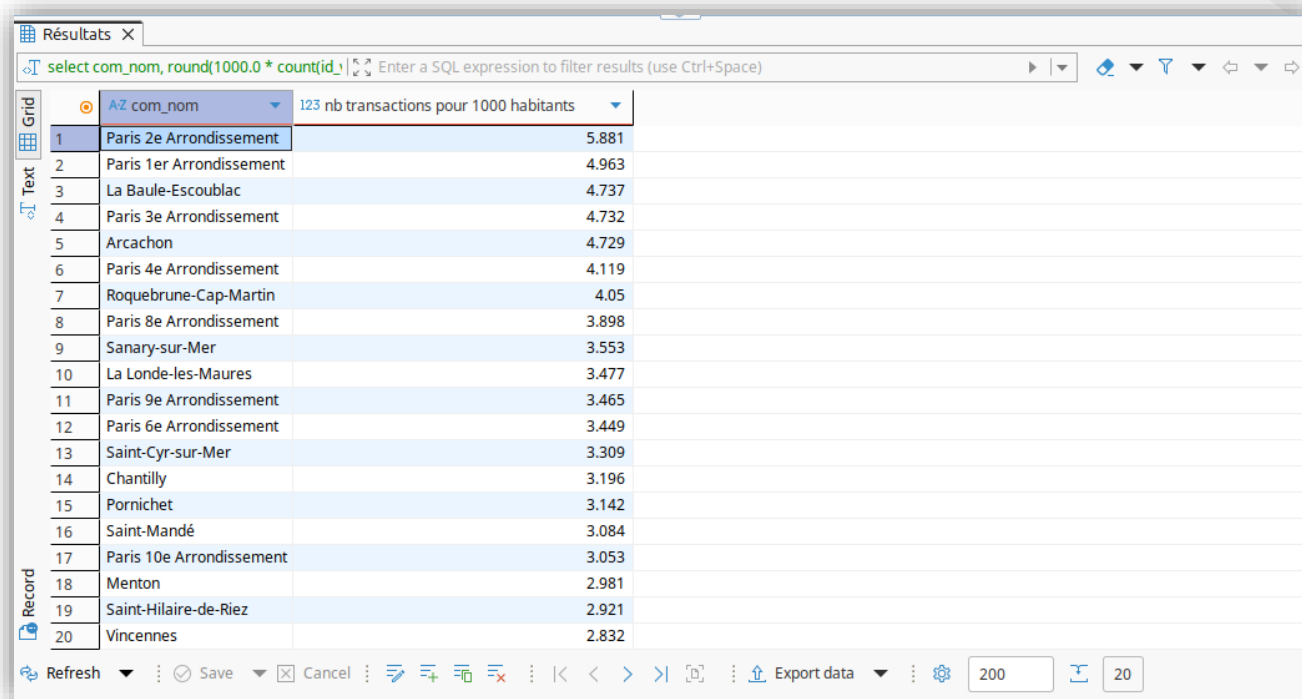
20 communes de plus de 10000 habitants avec le plus de transactions pour 1000 habitants

Requête 12

```
SELECT
  com_nom,
  ROUND(1000.0 * COUNT(id_vente) / population_municipale, 3)
  AS "nb transactions pour 1000 habitants"
FROM commune NATURAL LEFT JOIN bien
      NATURAL LEFT JOIN vente
WHERE population_municipale >= 10000
GROUP BY dep_code, com_code, com_nom
ORDER BY
  1000.0 * COUNT(id_vente) / population_municipale DESC,
  com_nom ASC, dep_code ASC, com_code ASC
LIMIT 20
```

20 communes de plus de 10000 habitants avec le plus de transactions pour 1000 habitants

Requête 12



The screenshot shows a database query results window titled "Résultats". The SQL query entered is `select com_nom, round(1000.0 * count(id_1))`. The results are displayed in a table with 20 rows, sorted by the transaction rate per 1000 inhabitants in descending order. The table has two columns: "com_nom" and "nb transactions pour 1000 habitants". The first row is "Paris 2e Arrondissement" with a value of 5.881, and the last row is "Vincennes" with a value of 2.832. The interface includes a sidebar with "Grid", "Text", and "Record" views, and a bottom toolbar with "Refresh", "Save", "Cancel", and "Export data" buttons.

	AZ com_nom	nb transactions pour 1000 habitants
1	Paris 2e Arrondissement	5.881
2	Paris 1er Arrondissement	4.963
3	La Baule-Escoublac	4.737
4	Paris 3e Arrondissement	4.732
5	Arcachon	4.729
6	Paris 4e Arrondissement	4.119
7	Roquebrune-Cap-Martin	4.05
8	Paris 8e Arrondissement	3.898
9	Sanary-sur-Mer	3.553
10	La Londe-les-Maures	3.477
11	Paris 9e Arrondissement	3.465
12	Paris 6e Arrondissement	3.449
13	Saint-Cyr-sur-Mer	3.309
14	Chantilly	3.196
15	Pornichet	3.142
16	Saint-Mandé	3.084
17	Paris 10e Arrondissement	3.053
18	Menton	2.981
19	Saint-Hilaire-de-Riez	2.921
20	Vincennes	2.832



Merci !